

Symfony 6 asociace ManyToMany

Obsah

1	Praktický návod.....	1
2	Teorie ManyToMany.....	7
3	Pro přidání dat do asociace ManyToMany mezi entitami v Symfony 6.....	9

1 Praktický návod

Zdroje

<https://webkul.com/blog/how-to-regenerate-entity-in-symfony-6-using-cli/>

<https://www.doctrine-project.org/projects/doctrine-orm/en/current/reference/working-with-objects.html#remove-object-many-to-many-join-tables>

<https://www.doctrine-project.org/projects/doctrine-orm/en/2.17/reference/association-mapping.html>

Vytvoříme asociaci ManyToMany pro tabulky Knihy, Autor

Vytvoříme nový projekt

symfony new manytomany --version="6.3.*" --webapp

Přidáme do projektu základní balíčky

composer require symfony/webpack-encore-bundle

npm install

npm run dev

npm run build

Vytvoříme entitu Autor (sloupec jmeno, prijmeni)

php bin/console make:entity Autor

Vytvoříme entitu Knihy

php bin/console make:entity Knihy

kde v průvodci příkazu zadáme název

```
New property name (press <return> to stop adding fields):  
> nazev
```

,... a přidáme atribut autor

```
Add another property? Enter the property name (or press <return> to stop adding fields):  
> autor
```

s datovým typem **relation**,

```
Field type (enter ? to see all types) [string]:  
> relation
```

zadáme relaci s Autor

```
What class should this entity be related to?:  
> Autor
```

dále v průvodci zadáme ManyToMany,

```
Relation type? [ManyToOne, OneToMany, ManyToMany, OneToOne]:  
> ManyToMany
```

Potvrdíme yes pro přístup z entity Autor do Knihy

```
Do you want to add a new property to Autor so that you can access/update Knihy objects from it - e.g. $autor->getKnihy s()? (yes/no) [yes]:  
>
```

Do entiy Knihy přidáme pod řádek ManyToMany následující řádek pro nastavení spojovací tabulky.

Pozn. Jde to i bez tohoto vložení řádku.

(Pomocí **JoinTable** určíte jméno spojovací tabulky, která bude obsahovat vazby mezi entitami.)

```
#[JoinTable(name: 'knihy_autori')]
```

Takto

```
#[ORM\ManyToMany(targetEntity: Autor::class, inversedBy: 'knihy')]  
#[JoinTable(name: 'knihy_autori')]
```

Zde je výpis celé entity Knihy

```
<?php

namespace App\Entity;

use App\Repository\KnihyRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: KnihyRepository::class)]
class Knihy
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $nazev = null;

    #[ORM\ManyToMany(targetEntity: Autor::class, inversedBy: 'knihy')]
    #[JoinTable(name: 'knihy_autori')]
    private Collection $autori;

    public function __construct()
    {
        $this->autori = new ArrayCollection();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getNazev(): ?string
    {
        return $this->nazev;
    }

    public function setNazev(string $nazev): static
    {
        $this->nazev = $nazev;

        return $this;
    }

    /**
     * @return Collection<int, Autor>
     */
    public function getAutori(): Collection
    {
        return $this->autori;
    }

    public function addAutori(Autor $autori): static
    {
        if (!$this->autori->contains($autori)) {
            $this->autori->add($autori);
        }
    }
}
```

```

        return $this;
    }

    public function removeAutori(Autor $autori): static
    {
        $this->autori->removeElement($autori);

        return $this;
    }
}

```

Zde je výpis celé entity Autor

```

<?php

namespace App\Entity;

use App\Repository\AutorRepository;
use Doctrine\Common\Collections\ArrayCollection;
use Doctrine\Common\Collections\Collection;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity(repositoryClass: AutorRepository::class)]
class Autor
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    #[ORM\Column(length: 255)]
    private ?string $jmeno = null;

    #[ORM\Column(length: 255)]
    private ?string $prijmeni = null;

    #[ORM\ManyToMany(targetEntity: Knihy::class, mappedBy: 'autori')]
    private Collection $knihy;

    public function __construct()
    {
        $this->knihy = new ArrayCollection();
    }

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getJmeno(): ?string
    {
        return $this->jmeno;
    }

    public function setJmeno(string $jmeno): static
    {
        $this->jmeno = $jmeno;

        return $this;
    }
}

```

```

public function getPrijmeni(): ?string
{
    return $this->prijmeni;
}

public function setPrijmeni(string $prijmeni): static
{
    $this->prijmeni = $prijmeni;

    return $this;
}

/**
 * @return Collection<int, Knihy>
 */
public function getKnihy(): Collection
{
    return $this->knihy;
}

public function addKnihy(Knihy $knihy): static
{
    if (!$this->knihy->contains($knihy)) {
        $this->knihy->add($knihy);
        $knihy->addAutori($this);
    }

    return $this;
}

public function removeKnihy(Knihy $knihy): static
{
    if ($this->knihy->removeElement($knihy)) {
        $knihy->removeAutori($this);
    }

    return $this;
}
}

```

V souboru .env nastavíme přístup do database mysql

```

DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?
serverVersion=mariadb-10.4.28"

```

Provedeme migraci

Tvorba migrace: **php bin/console make:migration**

Provedení migrace: **php bin/console doctrine:migrations:migrate --no-interaction**

V phpmyadmin můžeme přidat data

Autor

Struktura tabulky		Zobrazení relací							
#	Název	Typ	Porovnávání	Vlastnosti	Prázdný	Výchozí	Komentáře	Další	Operace
☐	1 id	int(11)			Ne	Žádná		AUTO_INCREMENT	Změnit
☐	2 jmeno	varchar(255)	utf8mb4_unicode_ci		Ne	Žádná			Změnit
☐	3 prijmeni	varchar(255)	utf8mb4_unicode_ci		Ne	Žádná			Změnit

knihy

#	Název	Typ	Porovnávání	Vlastnosti	Prázdný	Výchozí	Komentáře	Další
☐	1 id	int(11)			Ne	Žádná		AUTO_INCREMENT
☐	2 nazev	varchar(255)	utf8mb4_unicode_ci		Ne	Žádná		

propojovací tabulka knihy_autor

#	Název	Typ	Porovnávání	Vlastnosti	Prázdný	Výchozí	Komentáře	Další
☐	1 knihy_id	int(11)			Ne	Žádná		
☐	2 autor_id	int(11)			Ne	Žádná		

Po migraci se nastavily i relace v propojovací tabulce

FK_579F9EDF14D45BBE

Odstranit

ON DELETE

CASCADE

autor_id

+ Přidat sloupec

pokus

autor

id

ON UPDATE

RESTRICT

FK_579F9EDF3AA4C516

Odstranit

ON DELETE

CASCADE

knihy_id

+ Přidat sloupec

pokus

knihy

id

ON UPDATE

RESTRICT

V phpmyadmin vložíme do tabulek hodnoty

↩T→

▼

id

jmeno

prijmeni

☐

 Upravit

 Kopírovat

 Odstranit

1

Alois

Jirásek

☐

 Upravit

 Kopírovat

 Odstranit

2

Božena

Němcová

↩T→

▼

id

nazev

☐

 Upravit

 Kopírovat

 Odstranit

1

Babička

☐

 Upravit

 Kopírovat

 Odstranit

2

Psohlavci

Nyní vytvořme nový controller Hlavni

symfony console make:controller Hlavni

Překopírujeme zdrojový kód ze vzoru do controlleru Hlavni

```

class HlavniController extends AbstractController
{
    #[Route('/addkniha_autor/{knihaId}/{autorId}', name: 'addkniha_autor')]
    public function addknihaautor(EntityManagerInterface $entityManager,
    $knihaId, $autorId): Response
    {
        $kniha = $entityManager->getRepository(Knihy::class)-
>find($knihaId);
        $autor = $entityManager->getRepository(Autor::class)-
>find($autorId);

        if (!$kniha || !$autor) {
            throw $this->createNotFoundException('Kniha nebo autor nebyly
nalezeny');
        }

        // Přidání autora do knihy
        $autor->addKnihy($kniha);

        // Uložení změn v databázi
        $entityManager->flush();

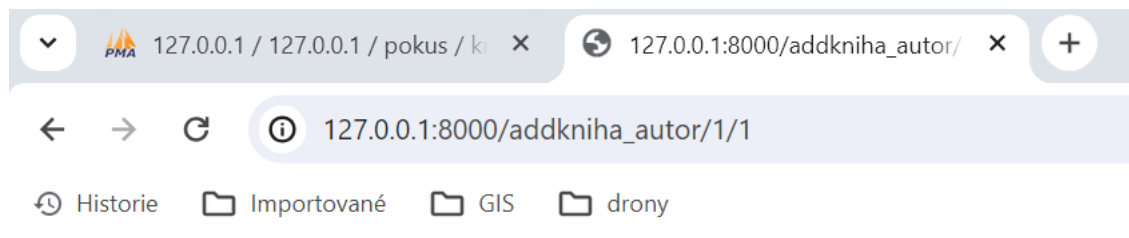
        return new Response('Knize byl přidán autor');
    }
}

```

Spustíme projekt

symfony server:start

Vložíme data



Knize byl přidán autor

2 Teorie **ManyToMany**

Zdroje

<https://webkul.com/blog/how-to-regenerate-entity-in-symfony-6-using-cli/>

<https://www.doctrine-project.org/projects/doctrine-orm/en/current/reference/working-with-objects.html#remove-object-many-to-many-join-tables>

<https://www.doctrine-project.org/projects/doctrine-orm/en/2.17/reference/association-mapping.html>

```

<?php
#[Entity]
class User
{
    // ...

    /**
     * Many Users have Many Groups.
     * @var Collection<int, Group>
     */
    #[ManyToMany(targetEntity: Group::class, inversedBy: 'users')]
    #[JoinTable(name: 'users_groups')]
    private Collection $groups;

    public function __construct() {
        $this->groups = new ArrayCollection();
    }

    // ...
}

#[Entity]
class Group
{
    // ...
    /**
     * Many Groups have Many Users.
     * @var Collection<int, User>
     */
    #[ManyToMany(targetEntity: User::class, mappedBy: 'groups')]
    private Collection $users;

    public function __construct() {
        $this->users = new ArrayCollection();
    }

    // ...
}

```


Vytvoříme entitu Product

```
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Product
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type: 'integer')]
    private $id;

    // Další vlastnosti produktu

    #[ORM\ManyToMany(targetEntity: Category::class, inversedBy:
"products")]
    #[ORM\JoinTable(name: "product_category")]
    private $categories;
}
```

Category

```
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Category
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type: 'integer')]
    private $id;

    // Další vlastnosti kategorie

    #[ORM\ManyToMany(targetEntity: Product::class, mappedBy:
"categories")]
    private $products;
}
```

Pomocí **JoinTable** určíte jméno spojovací tabulky, která bude obsahovat vazby mezi entitami.

Vytvoření getter a setter

symfony console make:entity --regenerate App\Entity\Kniha

3 Pro přidání dat do asociace `ManyToMany` mezi entitami v Symfony 6

můžete využít následující postup v controlleru:

Předpokládejme, že máme entity `Product` a `Category` s asociací `ManyToMany` mezi nimi. Chceme přidat existující produkt do existující kategorie.

Přidání produktu do kategorie v controlleru

```
use App\Entity\Category;
use App\Entity\Product;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class YourController extends AbstractController
{
    #[Route('/add-product-to-category/{productId}/{categoryId}', name:
    'add_product_to_category')]
    public function addProductToCategory(EntityManagerInterface $entityManager,
    $productId, $categoryId): Response
    {
        $product = $entityManager->getRepository(Product::class)->find($productId);
        $category = $entityManager->getRepository(Category::class)->find($categoryId);

        if (!$product || !$category) {
            throw $this->createNotFoundException('Produkt nebo kategorie nebyla
            nalezena');
        }

        // Přidání produktu do kategorie
```

```
$category->addProduct($product);

// Uložení změn v databázi
$entityManager->flush();

return new Response('Produkt byl přidán do kategorie');
}
}
```